Student Grade Management System

Introduction to Programming with Python

Group 2 (Presentation Group 4)

NINI Ulrich P. (11105723)
NOUDOFININ Priscillia L.B. (11044623)
OFIE Frédéric K. (10038624)
OKE Geoffroy M. (11053923)
OUBETTO Jean K. (10049424)

July 18, 2026

1 Introduction

1.1 Background and Problem Statement

In educational institutions, manually managing students' grades becomes increasingly difficult as the number of students grows. Our project aims to automate this process by developing a Python-based system that records student information, stores grades, automatically computes weighted averages, and generates statistical reports.

1.2 Project Objectives

- Design an intuitive graphical user interface for data entry.
- Implement a weighted average calculation system.
- Generate individual report cards and class statistics.
- Comply with the technical requirements (no object-oriented programming, only basic data structures).

2 Methodology

2.1 Problem Analysis

The system must manage:

- Four subjects with four assessments each.
- Different weight distributions (30% for E1/E2 and 20% for D1/D2).
- Missing grades.
- An overall ranking based on students' averages.

2.2 Algorithmic Choices

We chose to use:

- Nested dictionaries to store grades.
- Lists to manage student records.
- Modular functions for each operation.

```python
# Ranking display interface
def interface_afficher_classement():
    fen = tk.Toplevel(root)
    fen.title("Ranking")
    fen.geometry("600x400")
    tree = ttk.Treeview(fen,
                        columns=("name", "average"),
                        show="headings")
    tree.heading("average", text="Average")
    tree.pack(fill="both", expand=True)

# Average display interface
def interface_afficher_moyennes():
    fen = tk.Toplevel(root)
    fen.geometry("400x300")
    text = tk.Text(fen)
    text.pack(fill="both", expand=True)
```

Listing 1: Python Code Example

3 Implementation

3.1 Code Structure

- `mainloop()` : Manages the main menu.
- `display()` : Display functions.
- `calculate()` : Computation functions.

3.2 Challenges Encountered and Solutions

- Missing grades: handled using `None` values and filtering.
- Student ranking: implemented with `itemgetter`.
- Input validation: achieved through loops and exception handling.
- Development of the graphical interface using the tkinter library.

4 Results and Analysis

4.1 Results

The system provides:

- Unlimited student registration.
- Progressive grade entry.
- Automatic computation of class statistics.
- Generation of clear and readable reports.

4.2 Critical Analysis

- Strengths: intuitive graphical interface, modular code, robust error handling, and user-friendly design.
- Weaknesses: data is not permanently stored.

5 Conclusion

Our system successfully meets the project requirements while providing a solid foundation for future improvements. Possible enhancements include:

- Data persistence through file or database storage.
- Exporting reports and results to PDF format.